Multi-Cloud MCNF Cost Optimizer

Bjørn Remseth
Email: la3lma@gmail.com
With AI assistance

***Abstract*—Modern AI and data-intensive systems can generate heavy telemetry, training-data, simulation, analytics, and logging workloads whose costs span storage, ingress, egress, inter-cloud transfer, accelerator compute, artifact retention, and contractual discounts. This report develops a mathematical starting point for optimizing those workloads across multiple cloud providers. The central observation is that the placement and movement of data can be represented as a minimum-cost network flow problem when work can be split continuously, and as a mixed-integer linear program when data, jobs, or contractual commitments are indivisible. The resulting model gives buyers a disciplined way to compare vendor offers, quantify the real value of compute discounts, and expose cases where storage or egress penalties dominate apparently cheap accelerators.**

***Index Terms*—multi-cloud optimization, minimum-cost network flow, mixed-integer linear programming, cloud cost engineering, AI workload placement**

`version: 2026-06-24-17:31:32-CEST-839de3c-main`

Note on document creation: This document was created with AI assistance. See the "About This Document" section for details on the methodology and verification processes used.

I. Introduction

Multi-cloud cost optimization is a generic placement problem for data-intensive systems. An organization may need to collect telemetry, curate datasets, run AI training or analytics jobs, store model checkpoints, retain derived artifacts, and move selected outputs between providers or regions. Once the workload becomes large enough, the central cost question is no longer "which cloud has the cheapest GPU?" It becomes a routing and placement question:

> Where should each byte be stored, where should each computation run, where should each result live, and when is a vendor discount defeated by the cost of moving data?

The natural first abstraction is a minimum-cost network flow problem. Cloud providers, regions, storage classes, compute pools, and archival targets can be modeled as nodes in a directed graph. Data movement becomes flow. Storage, transfer, and compute prices become edge and node costs. Capacity, latency, governance, and contractual commitments become constraints.

The model below turns this placement problem into a working optimization structure and a practical data-collection plan. The goal is not to predict one universal cheapest cloud. The goal is to build a repeatable method that can answer specific workload questions with explicit assumptions.

II. Problem Scope

The first target workload is a generic heavy AI and data-processing pipeline:

- raw telemetry, logs, and source datasets ingested from upstream systems,
- storage of raw and curated data,
- transfer into a compute location,
- GPU or CPU processing for training, simulation, analytics, or batch inference,
- transfer of derived artifacts,
- long-term retention of models, reports, labels, and feature stores.

The relevant cloud-price components are dynamic and provider-specific. Official pricing pages for AWS, Google Cloud, Azure, and storage-specific alternatives such as Cloudflare R2 should be treated as source data rather than prose constants [1], [2], [3], [4], [5], [6], [7], [8], [9], [10]. The optimizer should ingest these prices from a maintained table with timestamps, regions, discount assumptions, storage classes, and currency normalization. In this first model, Cloudflare R2 is best represented as object storage with free outbound transfer from R2, while request-operation charges and Cloudflare edge compute products remain outside the model.

III. Network Representation

Let K be the set of feasible cloud locations. A location may be a provider-level abstraction such as AWS, Google Cloud, or Azure, but a production model should usually refine it to provider-region-service tuples:

$$K = \{(\text{provider}, \text{region}, \text{service class})\}.$$

For a single job or batch cycle, define the following workload constants:

- V^{raw}: raw input volume in GB.
- V^{out}: output artifact volume in GB.
- H: compute requirement, such as GPU-hours or CPU-hours.
- T^{raw}: raw-data retention duration in months.
- T^{out}: output-artifact retention duration in months.

Define provider-price parameters:

- I_i: ingress cost per GB into location i.
- S_i^{raw}: raw-data storage cost per GB-month at location i.
- S_i^{out}: artifact storage cost per GB-month at location i.
- C_j: compute cost per accelerator-hour or CPU-hour at location j.
- E_{ij}: transfer cost per GB from location i to location j.

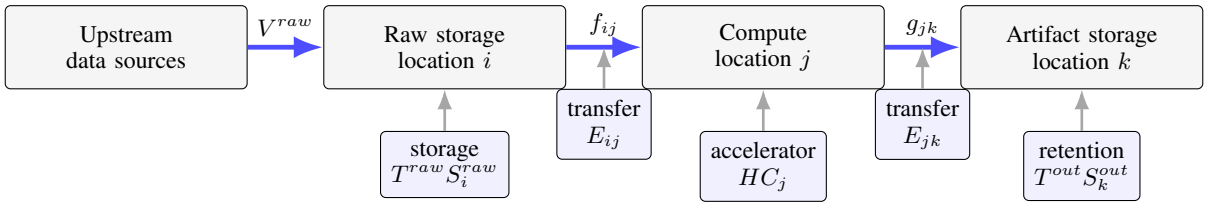

Fig. 1. *A multi-cloud workload as a placement-and-flow problem: raw data is stored, moved into compute, transformed, moved again, and retained as artifacts. Storage, compute, and transfer costs attach to different parts of the route.*

- A_j: available compute capacity at location j, measured in the same unit as H.

The diagonal entries E_{ii} are normally zero or near-zero for same-location transfer. This matters because egress costs create economic gravity: once data is placed in one cloud, the transfer matrix can make it expensive to exploit cheaper compute somewhere else.

IV. Pure Linear Model

When a workload can be split across locations, the decision variables can be continuous:

- $x_i \in [0,1]$: fraction of raw data stored at location i.
- $y_j \in [0,1]$: fraction of compute executed at location j.
- $z_k \in [0,1]$: fraction of output artifacts stored at location k.
- $f_{ij} \geq 0$: GB of raw data transferred from storage location i to compute location j.
- $g_{jk} \geq 0$: GB of output artifacts transferred from compute location j to storage location k.

The objective is to minimize total cost:

$$\begin{aligned}\min \quad & \sum_{i \in K} x_i V^{raw} (I_i + T^{raw} S_i^{raw}) \\ & + \sum_{i \in K} \sum_{j \in K} f_{ij} E_{ij} \\ & + \sum_{j \in K} y_j HC_j \\ & + \sum_{j \in K} \sum_{k \in K} g_{jk} E_{jk} \\ & + \sum_{k \in K} z_k V^{out} T^{out} S_k^{out}. \end{aligned} \tag{1}$$

The basic conservation constraints are:

$$\sum_{i \in K} x_i = 1, \quad \sum_{j \in K} y_j = 1, \quad \sum_{k \in K} z_k = 1, \tag{2}$$

$$\sum_{i \in K} f_{ij} = y_j V^{raw} \quad \forall j \in K, \tag{3}$$

$$\sum_{j \in K} g_{jk} = z_k V^{out} \quad \forall k \in K, \tag{4}$$

$$\sum_{j \in K} f_{ij} \leq x_i V^{raw} \quad \forall i \in K, \tag{5}$$

$$y_j H \leq A_j \quad \forall j \in K. \tag{6}$$

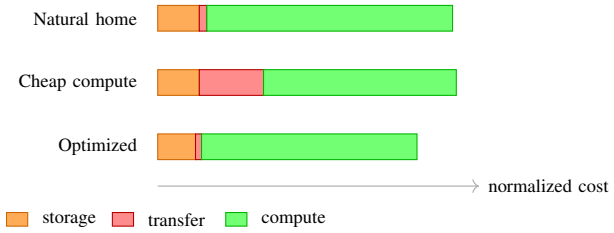

Fig. 2. *Illustrative cost decomposition for three allocation stories. The cheapest compute location is not necessarily cheapest overall once transfer and retained-storage terms are included.*

This is a linear program and can be solved with standard methods such as simplex or interior-point variants. Minimum-cost network flow is a deeply studied special case of linear optimization [11]. Python tooling such as SciPy's `linprog` can be used for the continuous form [12].

V. Mixed-Integer Variant

The pure LP is appropriate when a job is divisible: shards can be processed independently, partial datasets can train separate runs, and outputs can be combined later. Many practical workloads are not that clean. A training job may require the whole dataset in one place. A compliance rule may require one storage jurisdiction. A reserved-capacity contract may impose a minimum monthly spend. Those cases require integer or binary variables.

For all-or-nothing placement, replace x_i, y_j, and z_k with binary variables:

$$x_i, y_j, z_k \in \{0,1\}.$$

Then add the same sum-to-one constraints. This upgrades the formulation to a mixed-integer linear program. The model is computationally harder in theory, but for the small number of major cloud-provider and region combinations usually evaluated in procurement discussions, modern solvers can handle the first-order model quickly.

The mixed-integer form is also where vendor contracts become visible. For example, if a provider offers a discount only after a committed spend, introduce a binary activation variable q_p for provider p, a fixed or minimum-commitment term, and conditional cost coefficients. This turns vague procurement language into explicit constraints.

VI. Operational Data Model

The optimizer needs a workload table and a price table. The workload table should include:

- workload name and version,
- raw input volume distribution,
- output artifact volume distribution,
- compute-hour estimate by accelerator type,
- retention period by data class,
- divisibility flag: splittable, batch atomic, or pipeline-stage atomic,
- latency and deadline constraints,
- compliance constraints, such as region allow-lists.

The price table should include:

- provider, region, service, and SKU,
- currency and effective date,
- storage price by class and retention tier,
- compute price by instance or accelerator type,
- ingress and egress prices,
- negotiated discounts,
- committed-use terms,
- free-tier or bundled-transfer assumptions.

The important engineering rule is that price collection and optimization should be separated. Pricing pages are living documents; the optimization model should consume curated, auditable snapshots.

VII. Using the Model in Vendor Negotiation

The model turns vendor offers into falsifiable claims. Suppose a vendor offers a large GPU discount. That discount should not be evaluated in isolation. The new C_j value must be inserted into the full objective while preserving transfer costs E_{ij}, storage costs S_i, retention durations, and capacity constraints. A compute discount is only economically real if it survives the total-cost model.

The most useful negotiation outputs are:

- break-even egress price: the maximum transfer price at which remote compute remains attractive,
- break-even compute discount: the minimum discount needed to overcome data gravity,
- shadow price of capacity: the marginal value of one more accelerator-hour in a location,
- sensitivity to output volume: whether checkpoint-heavy workflows should compute near storage,
- switching threshold: the workload size at which a different placement becomes optimal.

These outputs are easier to discuss than raw cloud bills. They also shift the buyer's stance from anecdotal preference to mathematically supported procurement.

VIII. Prototype Optimizer

The repository includes a small prototype in a dedicated optimizer directory. The first version intentionally uses standard Python only and enumerates all single-placement choices for raw storage, compute, and output storage. It is not a replacement for a full LP or MILP solver, but it is useful for sanity-checking scenarios and explaining the cost decomposition. From the report directory, it can be run as:

```
python3 optimizer/mcnf_cost_optimizer.py
  optimizer/sample_scenario.json
```

A simple browser simulator accompanies the report and is published on the Journal of Bjorn as an interactive multi-cloud cost optimizer page. The simulator is illustrative rather than authoritative; it is intended to make the placement trade-offs visible while keeping the assumptions and price snapshot editable.

The complete prototype source appears in Appendix A. The next prototype should add a real LP backend for divisible workloads and a MILP backend for binary placement, capacity, and commitment constraints.

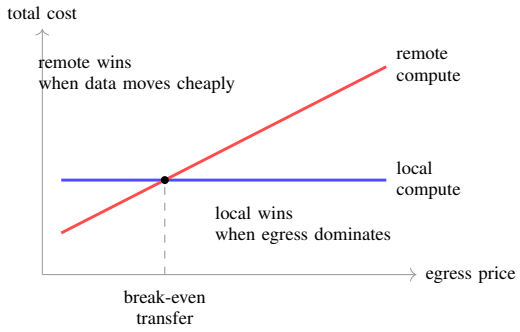

Fig. 3. *Sensitivity sketch for data gravity. A remote accelerator discount is economically meaningful only until transfer costs push the remote route above the local route.*

IX. Risks and Extensions

The simple model omits several important effects:

- tiered prices that change after volume thresholds,
- time-varying spot-market compute prices,
- reliability and retry costs,
- data locality constraints imposed by privacy or export-control rules,
- non-cloud costs such as engineering complexity and operational risk,
- pipeline DAGs with more than three stages,
- cache reuse across multiple jobs.

Most of these extensions still fit inside linear or mixed-integer linear optimization if represented carefully. Tiered pricing can be modeled with piecewise-linear segments. Pipeline DAGs can be represented with stage-indexed flows. Reliability constraints can be expressed as replication requirements. The danger is not mathematical impossibility; the danger is poorly maintained input data.

X. Conclusion

The cost of multi-cloud AI and data-intensive workloads can be formalized systematically. The cleanest starting point is a minimum-cost network flow model for divisible workloads, extended to mixed-integer linear programming when storage,

compute, contract, or compliance choices are indivisible. This gives the organization a repeatable way to compare providers, quantify data-gravity effects, and negotiate with vendors using explicit break-even conditions rather than intuition.

The immediate next step is to build a small data pipeline for provider price snapshots, then connect it to a solver-backed optimizer with scenario files for representative AI training, analytics, and artifact-retention workloads.

APPENDIX A
PROTOTYPE OPTIMIZER SOURCE

Listing 1. Complete prototype optimizer source.

```python
#!/usr/bin/env python3
"""Enumerate simple all-or-nothing multi-cloud placement choices.

This is a deliberately small prototype. It is useful for inspecting the cost
terms before replacing enumeration with a full LP or MILP solver.
"""

from __future__ import annotations

import argparse
import json
from dataclasses import dataclass
from itertools import product
from pathlib import Path
from typing import Any


@dataclass(frozen=True)
class Workload:
    name: str
    raw_gb: float
    output_gb: float
    compute_hours: float
    raw_retention_months: float
    output_retention_months: float


@dataclass(frozen=True)
class Location:
    name: str
    ingress_per_gb: float
    raw_storage_per_gb_month: float
    output_storage_per_gb_month: float
    compute_per_hour: float
    capacity_hours: float


@dataclass(frozen=True)
class Candidate:
    raw_location: str
    compute_location: str
    output_location: str
    total: float
    ingress: float
    raw_storage: float
    raw_transfer: float
    compute: float
    output_transfer: float
    output_storage: float


def require_float(data: dict[str, Any], key: str) -> float:
    try:
        return float(data[key])
    except KeyError as exc:
        raise ValueError(f"Missing required field: {key}") from exc


def load_scenario(path: Path) -> tuple[Workload, dict[str, Location], dict[str, dict[str, float]]]:
    data = json.loads(path.read_text(encoding="utf-8"))
    workload_data = data["workload"]
    workload = Workload(
        name=str(workload_data["name"]),
        raw_gb=require_float(workload_data, "raw_gb"),
        output_gb=require_float(workload_data, "output_gb"),
        compute_hours=require_float(workload_data, "compute_hours"),
        raw_retention_months=require_float(workload_data, "raw_retention_months"),
        output_retention_months=require_float(workload_data, "output_retention_months"),
    )

    locations = {
        name: Location(
            name=name,
            ingress_per_gb=require_float(values, "ingress_per_gb"),
            raw_storage_per_gb_month=require_float(values, "raw_storage_per_gb_month"),
            output_storage_per_gb_month=require_float(values, "output_storage_per_gb_month"),
            compute_per_hour=require_float(values, "compute_per_hour"),
            capacity_hours=require_float(values, "capacity_hours"),
```

```
        )
        for name, values in data["locations"].items()
    }

    egress = {
        source: {target: float(cost) for target, cost in targets.items()}
        for source, targets in data["egress_per_gb"].items()
    }
    return workload, locations, egress


def egress_cost(egress: dict[str, dict[str, float]], source: str, target: str) -> float:
    try:
        return egress[source][target]
    except KeyError as exc:
        raise ValueError(f"Missing egress cost from {source} to {target}") from exc


def enumerate_candidates(
    workload: Workload,
    locations: dict[str, Location],
    egress: dict[str, dict[str, float]],
) -> list[Candidate]:
    candidates: list[Candidate] = []
    names = sorted(locations)
    for raw_name, compute_name, output_name in product(names, repeat=3):
        raw = locations[raw_name]
        compute_loc = locations[compute_name]
        output = locations[output_name]

        if compute_loc.capacity_hours < workload.compute_hours:
            continue

        ingress = workload.raw_gb * raw.ingress_per_gb
        raw_storage = workload.raw_gb * workload.raw_retention_months * raw.raw_storage_per_gb_month
        raw_transfer = workload.raw_gb * egress_cost(egress, raw_name, compute_name)
        compute = workload.compute_hours * compute_loc.compute_per_hour
        output_transfer = workload.output_gb * egress_cost(egress, compute_name, output_name)
        output_storage = (
            workload.output_gb
            * workload.output_retention_months
            * output.output_storage_per_gb_month
        )
        total = ingress + raw_storage + raw_transfer + compute + output_transfer + output_storage

        candidates.append(
            Candidate(
                raw_location=raw_name,
                compute_location=compute_name,
                output_location=output_name,
                total=total,
                ingress=ingress,
                raw_storage=raw_storage,
                raw_transfer=raw_transfer,
                compute=compute,
                output_transfer=output_transfer,
                output_storage=output_storage,
            )
        )
    return sorted(candidates, key=lambda candidate: candidate.total)


def money(value: float) -> str:
    return f"${value:,.2f}"


def print_candidate(rank: int, candidate: Candidate) -> None:
    print(f"{rank}. total={money(candidate.total)}")
    print(f"   raw={candidate.raw_location}")
    print(f"   compute={candidate.compute_location}")
    print(f"   output={candidate.output_location}")
    print(
        "   terms: "
        f"ingress={money(candidate.ingress)}, "
        f"raw_storage={money(candidate.raw_storage)}, "
        f"raw_transfer={money(candidate.raw_transfer)}, "
        f"compute={money(candidate.compute)}, "
        f"output_transfer={money(candidate.output_transfer)}, "
        f"output_storage={money(candidate.output_storage)}"
    )


def main() -> int:
```

```
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("scenario", type=Path, help="Path to a scenario JSON file")
    parser.add_argument("--top", type=int, default=5, help="Number of candidates to print")
    args = parser.parse_args()

    workload, locations, egress = load_scenario(args.scenario)
    candidates = enumerate_candidates(workload, locations, egress)
    if not candidates:
        print("No feasible candidates found.")
        return 1

    print(f"Workload: {workload.name}")
    print(f"Feasible candidates: {len(candidates)}")
    print()
    for rank, candidate in enumerate(candidates[: args.top], start=1):
        print_candidate(rank, candidate)
    return 0


if __name__ == "__main__":
    raise SystemExit(main())
```

About This Document

This document represents a technical exploration created through a collaborative process between human and AI. The production process followed these steps:

1) **Discovery**: The scope was defined around cost control for telemetry, machine learning, storage, and transfer workloads across multiple clouds.
2) **Initial Exploration**: The problem was examined directly as a placement and routing question, leading to a minimum-cost network flow formulation and a mixed-integer variant for indivisible placement decisions.
3) **Synthesis**: The exploration was organized into a formal model with workload constants, decision variables, objective terms, constraints, operational data requirements, and vendor-negotiation outputs.
4) **Implementation**: A small standard-library Python prototype was added for exhaustive enumeration of single-placement scenarios. This is intended as a sanity-checking tool before introducing a full LP or MILP solver.
5) **Visualization**: Inline TikZ figures were added to show the workflow graph, cost decomposition, and data-gravity sensitivity. These figures are illustrative and are intended to explain the model rather than report measured benchmark results.
6) **Verification**: All references were scrutinized for authenticity. URLs were tested for accessibility, author names were verified where applicable, and content relevance was checked against citations. This verification process is documented in the following section.

The intended audience is technical leadership, cloud architects, platform engineers, data and AI infrastructure teams, and procurement stakeholders who need a shared mathematical language for cloud-cost decisions.

Note on References and Verification

This document contains AI-generated content. All references have been subject to rigorous verification to ensure academic integrity.

Verification Process:

- All URLs were tested for accessibility using automated tools
- Author names were verified against real publications where individual authors are cited
- DOIs were confirmed where available
- Publication venues and official documentation sources were validated
- Content relevance was checked against citations

Verification Status in References: Each reference includes a `note` field indicating its verification status:

- "Verified: URL accessible" – URL was tested and works
- "Verified: DOI accessible" – DOI was confirmed
- "Standard reference" – Well-known textbook or established work
- "Requires verification" – Needs manual review

Important Notice: Due to the AI-assisted nature of this document's creation, readers should independently verify any references used for critical applications. Cloud pricing is especially time-sensitive and should be refreshed before business decisions.

References

[1] Amazon Web Services, "Understanding data transfer charges," 2026, verified: URL accessible on 2026-06-24; official AWS Billing and Cost Management documentation describing data transfer charge categories. [Online]. Available: https://docs.aws.amazon.com/cur/latest/userguide/cur-data-transfers-charges.html

[2] ——, "Amazon s3 pricing," 2026, verified: URL accessible on 2026-06-24; official AWS pricing page for Amazon S3 storage, request, retrieval, and transfer cost components. [Online]. Available: https://aws.amazon.com/s3/pricing/

[3] Google Cloud, "Network pricing," 2026, verified: URL accessible on 2026-06-24; official Google Cloud network pricing documentation. [Online]. Available: https://cloud.google.com/vpc/network-pricing

[4] ——, "Cloud storage pricing," 2026, verified: URL accessible on 2026-06-24; official Google Cloud Storage pricing page. [Online]. Available: https://cloud.google.com/storage/pricing

[5] Microsoft Azure, "Bandwidth pricing," 2026, verified: URL accessible on 2026-06-24; official Microsoft Azure bandwidth pricing page covering data transfer in and out of Azure data centers. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/bandwidth/

[6] ——, "Azure blob storage pricing," 2026, verified: URL accessible on 2026-06-24; official Microsoft Azure Blob Storage pricing page. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/storage/blobs/

[7] Amazon Web Services, "Amazon ec2 on-demand instance pricing," 2026, verified: URL accessible on 2026-06-24; official AWS EC2 on-demand pricing page. [Online]. Available: https://aws.amazon.com/ec2/pricing/on-demand/

[8] Google Cloud, "Compute engine virtual machines pricing," 2026, verified: URL accessible on 2026-06-24; official Google Cloud Compute Engine pricing page, reached from vm-instance-pricing redirect. [Online]. Available: https://cloud.google.com/products/compute/pricing

[9] Microsoft Azure, "Linux virtual machines pricing," 2026, verified: URL accessible on 2026-06-24; official Microsoft Azure Linux Virtual Machines pricing page, reached from virtual-machines pricing redirect. [Online]. Available: https://azure.microsoft.com/en-us/pricing/details/virtual-machines/linux/

[10] Cloudflare, "R2 pricing," 2026, verified: URL accessible on 2026-06-24; official Cloudflare R2 pricing page listing standard storage, operation, retrieval, and egress pricing. [Online]. Available: https://developers.cloudflare.com/r2/pricing/

[11] R. K. Ahuja, T. L. Magnanti, and J. B. Orlin, *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall, 1993, standard reference: widely used textbook on network-flow theory and algorithms.

[12] SciPy Developers, "scipy.optimize.linprog," 2026, verified: URL accessible on 2026-06-24; official SciPy documentation for linear programming with linprog. [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.optimize.linprog.html